

Cheat Sheet: Python per LeetCode

Manuale di sopravvivenza essenziale per algoritmi e strutture dati.

Questo documento raccoglie tutta la sintassi, le strutture dati e i trucchi fondamentali per affrontare i problemi di LeetCode e la lista Grind 75 utilizzando Python 3.

1. Iterazioni e Sintassi di Base

Cicli (For e While)

In Python, si itera quasi sempre sugli elementi o tramite `range()`.

```
# Iterazione base su range (da 0 a 4)
for i in range(5):
    pass

# Iterare al contrario (da 4 a 0)
for i in range(4, -1, -1):
    pass

# Enumerazione: indice e valore simultaneamente
nums = [10, 20, 30]
for i, num in enumerate(nums):
    print(i, num)

# Iterare su due liste insieme (si ferma alla più corta)
for a, b in zip(listaA, listaB):
    pass
```

2. Strutture Dati (Le Armi Principali)

Liste (Array Dinamici) - $O(1)$ in fondo, $O(N)$ all'inizio

```
arr = []
arr.append(5)      # Aggiunge in fondo:  $O(1)$ 
arr.pop()          # Rimuove ultimo elemento:  $O(1)$ 
arr.insert(0, 9)   # Inserisce all'inizio:  $O(N)$  LENTO!

# Slicing (Creare sotto-liste)
sotto_lista = arr[1:3] # Indice 1 incluso, 3 escluso
copia = arr[::-1]      # Copia della lista invertita

# Inizializzare un array 1D (es. per Programmazione Dinamica)
dp = [0] * 10
```

Matrici (Griglie 2D)

Attenzione all'inizializzazione corretta per evitare alias in memoria!

```
R, C = 3, 4 # Righe e Colonne
# CORRETTO:
matrix = [[0] * C for _ in range(R)]

# SBAGLIATO (Crea riferimenti alla stessa riga):
# matrix = [[0] * C] * R
```

Dizionari (Hash Maps) - Lookup in O(1)

I dizionari sono la struttura più usata. Ideali per il pattern "Two Sum".

```
mappa = {}
mappa['chiave'] = 42

# Controllo di esistenza super veloce
if 'chiave' in mappa:
    del mappa['chiave'] # Rimuove la chiave

# Iterare su chiavi e valori
for chiave, valore in mappa.items():
    pass
```

Set (Insiemi) - Lookup in O(1)

Mantengono solo elementi unici. Ottimi per tracciare i nodi già visitati.

```
visti = set()
visti.add(5)
visti.remove(5) # Errore se non esiste, usa discard() per sicurezza

if 5 not in visti:
    print("Non ancora visitato!")
```

3. Librerie Avanzate (Essenziali per Grind 75)

Collections (Deque e Counter)

```
from collections import deque, Counter, defaultdict

# Deque (Coda Double-Ended): essenziale per la BFS
coda = deque([1, 2, 3])
coda.append(4)          # Aggiunge a destra: O(1)
coda.popleft()          # Rimuove da sinistra: O(1) (molto meglio di liste)

# Counter: conta le frequenze in una riga
conteggio = Counter([1, 1, 2, 3]) # {1: 2, 2: 1, 3: 1}

# DefaultDict: evita KeyError se la chiave non esiste
adj = defaultdict(list)
adj[1].append(2) # Se 1 non esiste, inizializza automaticamente []
```

Heapq (Code di Priorità / Min-Heap) - Inserimento $O(\log N)$

Python implementa nativamente solo Min-Heap. Per Max-Heap, inserisci i valori negativi.

```
import heapq

min_heap = []
heapq.heappush(min_heap, 5)
heapq.heappush(min_heap, 2)
minimo = heapq.heappop(min_heap) # Estrae 2

# Trasformare una lista esistente in un Heap:  $O(N)$ 
numeri = [5, 7, 1, 3]
heapq.heapify(numeri)
```

4. Trucchi e Funzioni Integrate

Ordinamento e Lambda

```
# Ordina l'array originale
arr.sort()

# Ritorna un nuovo array ordinato
nuovo_arr = sorted(arr, reverse=True)

# Ordinamento custom (es. per il secondo elemento)
intervalli = [[1, 3], [2, 6], [8, 10]]
intervalli.sort(key=lambda x: x[1])
```

Valori Infiniti, Max/Min, Swap

```
# Swap tra due variabili
a, b = b, a

# Inizializzare minimi o massimi assoluti
max_val = float('-inf')
min_val = float('inf')

# Aggiornare con min/max inline
risultato = max(risultato, nuovo_valore)
```

Stringhe e conversioni

Le stringhe in Python sono immutabili (non puoi fare `s[0] = 'a'`). Per manipolarle, converti in lista e poi riunisci.

```
s = "ciao"
lista_caratteri = list(s)      # ['c', 'i', 'a', 'o']
lista_caratteri[0] = 'm'
s_modificata = "".join(lista_caratteri) # "miao"

# ASCII conversioni (utili nei grafi di stringhe)
codice = ord('a')  # Ritorna 97
carattere = chr(97) # Ritorna 'a'
```